OptiCrop: Smart Agricultural Production Optimization Engine

Project Description:

OptiCrop harnesses Machine Learning to provide intelligent agricultural crop recommendations, offering farmers a fast and data-driven decision-making experience. The platform includes a Crop Recommendation module, which predicts the most suitable crop based on soil and environmental inputs, a Crop Suitability Assessment module, which evaluates whether existing field conditions match a selected crop's ideal requirements, and a Research and Analytics Dashboard, which presents model performance metrics and feature importances to agricultural researchers and policymakers.

Utilizing a Random Forest classifier trained on 2,760 records across 23 crop types, OptiCrop processes seven key input parameters — Nitrogen (N), Phosphorous (P), Potassium (K), Temperature, Humidity, pH, and Rainfall — to deliver accurate crop recommendations achieving 92.57% accuracy. Built with Flask and deployed on Vercel, the platform ensures a seamless and user-friendly experience. With a clean Bootstrap 5 interface and a REST API for programmatic access, OptiCrop empowers farmers, agribusinesses, researchers, and policymakers to make evidence-based farming decisions with confidence.

Scenarios

Scenario 1: A farmer enters soil nutrient values (Nitrogen, Phosphorous, Potassium) and climate conditions (Temperature, Humidity, pH, Rainfall) into the Crop Recommendation page. The system analyzes the data using the trained Random Forest model and recommends the most suitable crop — such as Rice or Wheat — with a confidence percentage, a top-3 probability bar chart, and a practical farming tip, all in seconds.

Scenario 2: A farmer already planning to grow a specific crop selects it from the Suitability Assessment page and enters current field conditions. OptiCrop evaluates each of the 7 parameters against ideal agronomic ranges stored in the model metadata and returns a 0–100% suitability score displayed as a circular gauge, along with specific parameter gap warnings and improvement recommendations.

Scenario 3: An agricultural researcher or policymaker uses the Research and Analytics Dashboard to explore crop-environment relationships. The dashboard presents a Chart.js powered model comparison bar chart, a feature importance doughnut chart, a scrollable crop-environment profiles reference table for all 23 crops, and a REST API reference for programmatic integration with external agricultural systems.

Technical Architecture:

Description: OptiCrop utilizes a modular three-tier architecture to deliver rapid, ML-driven crop recommendations. The frontend provides a responsive user interface built with HTML, CSS, and Bootstrap 5 while the Flask-based backend orchestrates input validation, feature scaling, and model inference. It interfaces with the trained Random Forest model serialized as a Pickle file to generate tailored crop recommendations, with secure deployment managed on Vercel and an open REST API endpoint for external integrations.

(Note: If you are using a database in your project you definitely need to add an ER Diagram along with description)

Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation
- **Scikit-learn & ML Concepts:** Scikit-learn Documentation
- **HTML, CSS, and Bootstrap 5 Skills:** Bootstrap 5 Documentation
- **Python Programming Proficiency:** Python Documentation
- **Pandas & NumPy for Data Handling:** Pandas Documentation
- **Version Control with Git:** Git Documentation
- **Vercel for Cloud Deployment:** Vercel Documentation

Project Workflow:

Milestone 1: Environment Setup & Model Selection

- **Activity 1.1:** Set up the Python virtual environment and install all required dependencies including Flask, scikit-learn, pandas, numpy, matplotlib, seaborn, and Jupyter.
- **Activity 1.2:** Research and select the appropriate machine learning algorithm from scikit-learn for crop classification across 23 crop types.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, ML model, and data pipeline.
- **Activity 1.4:** Configure the project directory structure including folders for templates, static files, model artifacts, and data.

Milestone 2: Machine Learning Model Development

- **Activity 2.1:** Analyze and preprocess the agricultural dataset; implement feature scaling using StandardScaler and label encoding using LabelEncoder.
- **Activity 2.2:** Train and evaluate four ML algorithms — KNN, Logistic Regression, Decision Tree, and Random Forest — selecting the best-performing model.

Milestone 3: Core Backend Development (app.py)

- **Activity 3.1:** Develop the main Flask application logic in app.py establishing five routes: /, /predict, /suitability, /research, and /api/predict.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the responsive web interface using HTML, CSS, and Bootstrap 5, creating four page templates: index.html, predict.html, suitability.html, and research.html.
- **Activity 4.2:** Integrate Chart.js for the Research Dashboard, implement the SVG circular suitability gauge, and create dynamic Jinja2 templates to display prediction results.

Milestone 5: Testing & Deployment

- **Activity 5.1:** Perform model evaluation and system testing to ensure accurate crop recommendations; validate the Flask application locally using the development server.

- **Activity 5.2:** Deploy the application on Vercel using a vercel.json configuration file to expose the Flask app publicly at a secure cloud URL.

Milestone 6: Conclusion

Milestone 1: Environment Setup & Model Selection

In this milestone, we focus on setting up the development environment and selecting the appropriate machine learning model for OptiCrop. This involves installing all required Python libraries, configuring the project structure, and researching the capabilities and performance of various ML algorithms from scikit-learn, ensuring the chosen model aligns well with the application's objective of classifying crops across 23 types based on 7 soil and environmental input parameters.

Activity 1.1: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv venv
D:\projects>"d:\projects\venv\Scripts\activate"
(venv) D:\projects> pip install -r requirements.txt
```

- **Install Required Libraries:** Install all ML and web framework dependencies:

```
(venv) D:\projects> pip install Flask>=2.3.0
(venv) D:\projects> pip install scikit-learn>=1.3.0
(venv) D:\projects> pip install pandas>=2.0.0 numpy>=1.24.0 (venv)
D:\projects> pip install matplotlib>=3.7.0 seaborn>=0.12.0
(venv) D:\projects> pip install jupyter>=1.0.0
```

- **Set Up Application Structure:** Create the project directory structure:

```
OptiCrop/ app.py # Flask application
requirements.txt vercel.json # Vercel
deployment config model/ train_models.py # ML
training pipeline crop_model.pkl # Trained
Random Forest (92.57%) scaler.pkl #
StandardScaler label_encoder.pkl # LabelEncoder
```

```
(23 classes) model_meta.json # Metrics + crop
profiles data/ crop_data.csv # 2,760-record
dataset templates/ base.html index.html
predict.html suitability.html research.html
static/css/ static/js/
```

Activity 1.2: Research and Select the Appropriate ML Algorithm

Here are the things needed to research in order to select the best model for the project:

- **Understand the Project Requirements:** Review the specific needs of OptiCrop — classifying crops from 7 continuous numerical inputs across 23 target classes.
- **Explore Scikit-learn's Algorithm Documentation:** Examine available classifiers including KNN, Logistic Regression, Decision Tree, Random Forest, and K-Means Clustering.
- **Evaluate Model Performance:** Compare algorithms based on accuracy, F1 score, and 5-fold cross-validation score using the 2,760-record training dataset.
- **Select the Optimal Model:** Choose **Random Forest (n_estimators=100)** for its superior accuracy of 92.57%, F1 score of 92.48%, and cross-validation score of 91.85%, outperforming all other evaluated algorithms.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture including components: Bootstrap 5 frontend, Flask backend, ML model pipeline, and Vercel deployment.
- **Detail Frontend Functionality:** Outline how users interact with the application — entering soil NPK values and climate conditions, selecting scenario pages (Recommend / Suitability / Research), and viewing AI-generated results.
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming GET and POST requests for each route, validates and preprocesses user input through the StandardScaler, calls predict_proba() on the loaded Random Forest model, and returns structured results to the Jinja2 templates.
- **Describe ML Pipeline Integration:** Define how the backend loads pre-trained artifacts (crop_model.pkl, scaler.pkl, label_encoder.pkl, model_meta.json) at startup using pickle.load(), processes inputs through the same scaler used during training, and maps predicted integer classes back to crop names using the LabelEncoder.

Milestone 2: Machine Learning Model Development

Activity 2.1: Dataset Generation & Preprocessing

Generate and preprocess the agricultural dataset used to train all four ML models.

- **Dataset Generation:** Generate 2,760 records (120 samples per crop × 23 crops) using agronomically realistic parameter ranges defined in the CROP_PROFILES dictionary — covering N, P, K, temperature, humidity, pH, and rainfall for each crop.
- **Feature Scaling:** Apply StandardScaler (zero-mean, unit-variance) to all 7 numerical features. The fitted scaler is saved as scaler.pkl and must be reused identically during inference in app.py to prevent data leakage.
- **Label Encoding:** Convert 23 string crop labels (e.g., 'Rice', 'Wheat') to integer class indices (0–22) using LabelEncoder. Saved as label_encoder.pkl for decoding predictions back to crop names.
- **Train-Test Split:** Split dataset 80/20 (2,208 train / 552 test) with stratify=y to maintain class balance across both sets.

Activity 2.2: Train & Evaluate Four ML Algorithms

Train and compare all four classifiers using identical train/test splits and evaluate with accuracy, weighted F1, and 5-fold cross-validation:

- **Save Best Model:** Serialize the Random Forest model, scaler, and label encoder using pickle.dump() and save model metrics and 23 crop profiles to model_meta.json for use by the suitability assessment and research dashboard pages.

```
pickle.dump(best_model, open('model/crop_model.pkl', 'wb')) # RF 92.57%
pickle.dump(scaler, open('model/scaler.pkl', 'wb')) # StandardScaler
pickle.dump(le, open('model/label_encoder.pkl', 'wb')) # 23 classes
json.dump(meta, open('model/model_meta.json', 'w')) # metrics + profiles
# Also save K-Means clustering model for research analytics:
pickle.dump(KMeans(n_clusters=23).fit(X), open('model/kmeans.pkl','wb'))
```

Milestone 3: Core Backend Development (app.py)

Milestone 3 focuses on creating the core logic of the app.py file, which serves as the backbone of the OptiCrop application. In this milestone, you will set up all five Flask routes, implement the prediction and suitability scoring functions, and establish reliable model loading at startup.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- Establish routes for OptiCrop's five endpoints, each serving a distinct purpose:
  - / — Home page route that renders the index.html dashboard

```
@app.route("/")
def index():
return render_template("index.html", meta=meta)
```

  - **/predict** — GET+POST endpoint that accepts soil/climate form data, runs the Random Forest model, and returns top-3 crop recommendations

```
@app.route("/predict",
methods=["GET","POST"]) def predict(): result
= None if request.method == 'POST':
inputs = np.array([[ float(request.form['nitrogen']),
float(request.form['phosphorous']),
float(request.form['potassium']),
float(request.form['temperature']),
float(request.form['humidity']), float(request.form['ph']),
float(request.form['rainfall']), ]]) scaled =
scaler.transform(inputs) # StandardScale proba =
model.predict_proba(scaled)[0] # 23 probabilities top3 =
np.argsort(proba)[::-1][:3] # top 3 indices crop =
le.classes_[top3[0]] # e.g. 'Rice' conf = round(proba[top3[0]]
* 100, 1) # e.g. 56.0% return render_template('predict.html',
result=result, meta=meta)
```

  - **/suitability** — POST endpoint that scores each input parameter against ideal crop ranges and returns a 0–100% compatibility gauge
  - **/research** — GET endpoint rendering the analytics dashboard with Chart.js model comparison and feature importance charts
  - **/api/predict** — JSON REST API endpoint returning structured prediction results for programmatic integration

Define Prediction & Suitability Helper Functions:

- **load_artifacts():** Loads all four pickle/JSON model files at Flask startup. Must use the same scaler fitted on the training set to avoid data leakage during inference.

```
def load_artifacts():
model = pickle.load(open('model/crop_model.pkl', 'rb')) # RF scaler =
pickle.load(open('model/scaler.pkl', 'rb')) # scaler le =
pickle.load(open('model/label_encoder.pkl', 'rb')) # encoder meta =
json.load(open('model/model_meta.json')) # profiles return model, scaler,
le, meta

model, scaler, le, meta = load_artifacts() # run once at startup
```

- **CROP_ICONS & CROP_TIPS:** Dictionaries mapping crop names to display emoji icons and farming tip strings rendered in the predict.html result panel.
- **assess_suitability():** Compares each input value against the ideal range stored in model_meta.json for the selected crop. Scores each parameter 0.0–1.0 and returns a weighted average as the overall suitability percentage.

```
def assess_suitability(crop, inputs):
profiles = meta['crop_profiles'] profile_keys =
['N','P','K','temp','hum','ph','rain'] feat_names =
['N','P','K','temperature','humidity','ph','rainfall'] vals =
inputs[0] issues, scores = [], [] for i, (feat, pkey) in
enumerate(zip(feat_names, profile_keys)):
lo, hi =
profiles[crop][pkey] v =
vals[i] if lo <= v <= hi:
scores.append(1.0) # within ideal range
else:
gap = min(abs(v-lo), abs(v-hi)) penalty = min(gap / (hi-lo+1e-9),
1.0) scores.append(max(0, 1-penalty)) direction = 'low' if v < lo
else 'high' issues.append(f'{feat} is {direction} ({v:.1f}; ideal
{lo}-{hi})') return round(np.mean(scores)*100, 1), issues
```

Set Up Route Handlers for Each Feature:

- For the /predict route, implement logic that reads 7 form fields, scales inputs via scaler.transform(),runs model.predict_proba() to get 23 class probabilities, sorts descending, and passes top-3 results to predict.html.

- For the /suitability route, read the selected crop name and 7 field inputs, call assess_suitability(), andreturn the score, level label (Excellent/Good/Moderate/Poor), and issue list to suitability.html.
- For the /api/predict route, accept JSON body via request.get_json(), run the same predictionpipeline, and return {crop, confidence, top3} as a JSON response.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for OptiCrop. This involves designing a clean agricultural-themed layout with responsive HTML, CSS, and Bootstrap 5, and creating dynamic content rendering with Jinja2 templates and Chart.js to display ML-generated crop recommendations and analytics.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the base.html template as the shared layout for all pages, including a green-themedBootstrap 5 navbar showing OptiCrop brand, four navigation links (Home, Recommend, Suitability, Research), and a footer displaying the deployed model name and accuracy.
- Use Jinja2 block inheritance — {% extends 'base.html' %} and {% block content %} — to maintain aconsistent layout across all five pages while allowing page-specific content.
- Integrate Bootstrap Icons and Google Fonts (Inter) for clean typography and iconography

Design a Responsive Layout Using CSS:

- Create CSS custom properties for the agricultural color palette: --green-dark (#1a4731), --green-mid(#2d7a4f), --green-light (#4aad73), used consistently for navbars, buttons, result cards, and badges.
- Use Bootstrap 5 grid and flexbox layouts to organize the predict page into a two-column structure:input form on the left and result panel on the right.
- Add Bootstrap responsive breakpoints to ensure the layout adapts gracefully across desktop,

tablet,and mobile screen sizes.

Create Separate UI Components for Each Output Type:

- **Result Hero Card:** Gradient green card displaying the recommended crop emoji icon, crop name, and confidence percentage rendered conditionally using {% if result %}.
- **Top-3 Progress Bars:** Animated Bootstrap progress bars for each of the three most likely crops, with width set dynamically via Jinja2: style='width:{{ prob }}%;'.
- **Suitability Gauge:** SVG circular gauge with stroke-dasharray calculated dynamically: {{ (result.suitability / 100 * 339.3)|round(1) }} 339.3, changing color from green (Excellent) to amber (Good) to red (Poor).
- **Chart.js Charts:** Research page embeds a bar chart comparing 4 model accuracies and a doughnut chart showing feature importances, both populated with data passed from Flask via {{ meta | tojson }}.

Activity 4.2: Creating Dynamic Content Rendering with Jinja2 & JavaScript

Handle Form Submission:

- Predict and suitability pages use standard HTML form POST submissions (method='POST')processed directly by Flask, with results rendered server-side by Jinja2 templates — no AJAX required.
- Quick Fill preset buttons on the predict page use JavaScript onclick handlers to populate all 7 formfields instantly with sample values for Rice, Wheat, Maize, Coffee, and Cotton.

Render Results Dynamically with Jinja2:

- Use {% if result and not result.error %} conditional blocks to show the result panel only after asuccessful POST submission, keeping the initial GET page state clean.
- After form submission, Bootstrap's scroll behavior and page anchor links automatically position theviewport to display the result panel at the top.

Research Dashboard — Chart.js Integration:

```
<!-- research.html — Pass Flask data to Chart.js -->
<script> const meta = {{ meta | tojson }}; // Flask →
JS bridge

// Model comparison bar chart new
Chart(document.getElementById('modelChart'), { type: 'bar',
data: { labels: ['KNN','Logistic Reg','Decision Tree','Random
Forest'], datasets: [ { label:'Accuracy %', data:[85.87,
84.96, 89.31, 92.57] }, { label:'F1 Score %', data:[85.68,
84.78, 89.34, 92.48] },
]
}
});

// Feature importance doughnut chart new
Chart(document.getElementById('featureChart'), { type:
'doughnut', data: { labels:
Object.keys(meta.feature_importances), datasets: [{ data:
Object.values(meta.feature_importances) }] }
});
</script>
```

Milestone 5: Testing & Deployment

In Milestone 5, the focus is on testing the OptiCrop application locally to verify correct ML predictions and UI rendering, and then deploying the application publicly on Vercel to make it accessible from anywhere. This involves configuring the Vercel deployment, managing model artifact storage, and validating the full prediction pipeline end-to-end.

Activity 5.1: Training the Model & Local Testing

Train the ML Pipeline:

- Run the training pipeline to generate the dataset, train all 4 models, and save all artifacts:

```
# Step 1: Train all models and save artifacts
(venv) D:\projects> python model/train_models.py

# Expected output:
# [1/5] Generating dataset ... 2760 records | 23
crops # [2/5] Preprocessing ... Train: 2208 | Test:
552 # [3/5] Training models ...
# KNN | Acc: 85.87% F1: 85.68% CV: 85.87%
# Logistic Regression | Acc: 84.96% F1: 84.78% CV: 87.41%
# Decision Tree | Acc: 89.31% F1: 89.34% CV: 88.27%
# Random Forest | Acc: 92.57% F1: 92.48% CV: 91.85%
# [4/5] Best model: Random Forest (92.57%)
# [5/5] Saving: crop_model.pkl / scaler.pkl / label_encoder.pkl / model_meta.json
```

Start the Flask Development Server:

- Run the application locally using:

```
(venv) D:\projects> python app.py

* Serving Flask app 'app'
* Debug mode: on* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

- Once running, open a browser and navigate to http://127.0.0.1:5000 to access the OptiCropapplication.
- Test all three scenarios — enter sample soil/climate values on the Predict page, check suitability fordifferent crops on the Suitability page, and verify Chart.js charts load correctly on the Research page.

Activity 5.2: Cloud Deployment on Vercel

Vercel provides serverless Python deployment, making the OptiCrop Flask app accessible from anywhere with a secure HTTPS URL without requiring a dedicated server.

Configure vercel.json:

- Create a vercel.json file in the project root to configure the Vercel Python runtime:

```
// vercel.json — Vercel deployment configuration
{
"version": 2,
"builds": [ { "src": "app.py", "use":
"@vercel/python" }
],
"routes": [ { "src": "/(.*)", "dest":
"app.py" }
]
}
```

Deploy to Vercel:

```
# Install Vercel CLI D:\projects>
npm install -g vercel

# Login and deploy
D:\projects> vercel login
D:\projects> vercel --prod

# Output:
# Deployed to: https://opti-crop-xxx-sai-charans-projects.vercel.app
```

- The Vercel deployment automatically installs all dependencies from requirements.txt, runs the Flaskapp using the Python runtime, and exposes all routes including the REST API endpoint /api/predict.
- **Important Note:** Pickle model files (.pkl) must be committed to the Git repository so Vercel can access them at runtime. Do not add .pkl files to .gitignore for this project. • **Live URL:** opti-crop-nux2ld79v-s-sai-charans-projects.vercel.app

Exploring the Website's Web Pages:

Home Page (/):

Description: The Home page of OptiCrop features a hero section with a green gradient background displaying the project title, tagline, and four key statistics (Model Accuracy, Crop Types, Parameters, Training Records). Below the hero are three scenario cards explaining each use case, a model performance comparison section showing all four algorithm results, and a 7-parameter reference section.

The Bootstrap 5 navbar at the top provides navigation to all four pages.

Crop Recommendation Page (/predict):

Description: The Predict page is Scenario 1 — the core farmer-facing interface. The left panel contains a structured Bootstrap 5 form with inputs grouped into Soil Nutrients (N, P, K in mg/kg) and Climate Conditions (Temperature in °C, Humidity %, pH 0–14, Rainfall in mm). Quick-fill preset buttons populate all fields for common crops. On form submission, Flask routes the POST request to the prediction pipeline and re-renders the page with the result panel on the right — showing the recommended crop icon, name, confidence percentage, animated top-3 probability bars, an input summary grid, and a farming tip card.

Suitability Assessment Page (/suitability):

Description: The Suitability page is Scenario 2. Users select a target crop from a dropdown menu populated with all 23 crop names from model_meta.json, then enter their current field conditions. Upon submission, the assess_suitability() function compares each parameter against stored ideal ranges and returns a percentage score rendered as an SVG circular gauge with color-coded levels (Excellent ≥80%, Good 60–80%, Moderate 40–60%, Poor <40%). Parameter gaps are listed as amber warning alerts with specific adjustment advice. A live ideal ranges reference table auto-populates when a crop is selected via JavaScript.

Research & Analytics Dashboard (/research):

Description: The Research page is Scenario 3, designed for agricultural researchers, agribusinesses, and policymakers. The page opens with four KPI stat cards showing model accuracy, crop types, input parameters, and number of algorithms compared. Below are two Chart.js charts — a grouped bar chart comparing Accuracy, F1, and Cross-Validation scores for all four ML algorithms, and a doughnut chart visualizing Random Forest feature importances (Phosphorous 18.86%, Rainfall 16.23%, Nitrogen 16.14%). A scrollable crop environment profiles table lists ideal parameter ranges for all 23 crops. A REST API reference panel at the bottom shows how to call the /api/predict JSON endpoint programmatically.

Conclusion:

OptiCrop is a complete machine learning-powered agricultural optimization system built with Flask and a clean Bootstrap 5 interface that simplifies data-driven crop decision-making for farmers, researchers, and policymakers. It uses a trained Random Forest classifier achieving 92.57% accuracy to instantly recommend the most suitable crop from 23 types based on 7 soil and climate parameters, assess suitability scores with parameter-level feedback, and present model analytics and crop-environment profiles through an interactive Research Dashboard. The project — which included dataset generation, ML model training and evaluation across four algorithms, Flask backend development, Jinja2 frontend templating, Chart.js integration, and Vercel cloud deployment — demonstrates how targeted machine learning and a structured web framework can significantly boost agricultural productivity and resource efficiency. Looking ahead, OptiCrop offers significant opportunities for expansion, such as integrating real-time weather API data for dynamic recommendations, adding image-based soil analysis using computer vision, supporting regional language interfaces for rural farmer accessibility, and implementing user authentication to save field history and track recommendation outcomes over crop seasons. By continuously improving the underlying ML model with real-world field data and enhancing the frontend experience, OptiCrop is well-positioned to evolve from a student project into a production-grade agricultural intelligence platform serving farmers at scale.